Data Handling: Import, Cleaning and Visualisation

Lecture 7:
Data Preparation, Manipulation, and Cleaning I

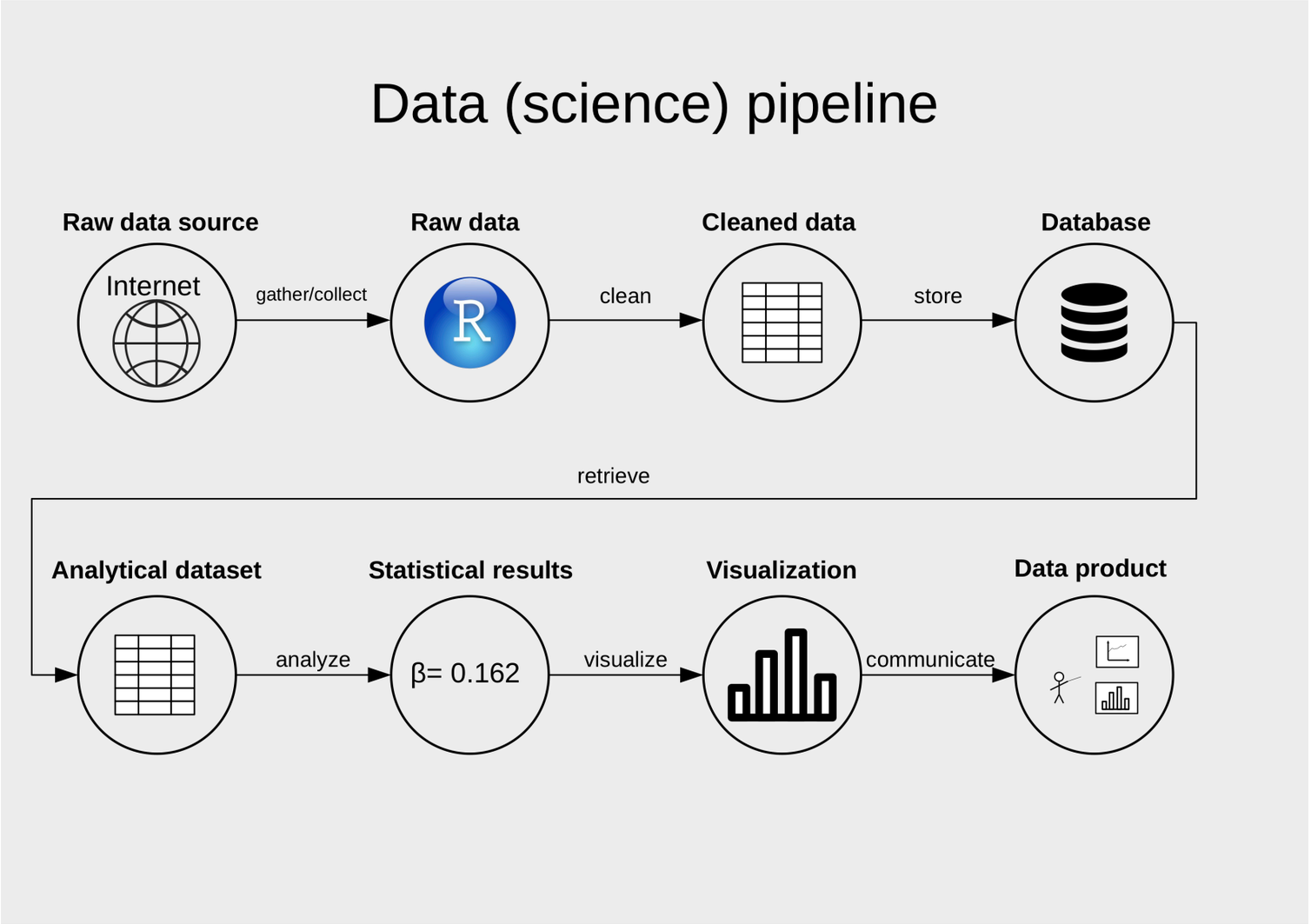
Dr. Aurélien Sallin

2025-11-13

Welcome back!

Recap of Part I

Part I: Data, data processing, data structures



Part I: Data, data processing, data structures

Our story so far:

- 🖥️ Computers store data as bits (Bits → Bytes → Encoded Text)
- ✍️ Text encodes meaning
- 📝 Files encode structure (→ Structured Files like CSV, JSON, XML)
- 🗂️ Structure is more or less flexible (→ rectangular data, hierarchical data)
- 🔄 ✋ Structure is obtained automatically or manually (→ APIs vs local files)
- 💾 R parses structure into memory. (→ R Parsing → R Data Structures)

Part I: Your own Group Project

🎉

Part I: Your own Group Project

Recap: non-rectangular data

XML vs JSON: Structure and Syntax

XML:

```xml
<person>
  <firstName>John</firstName>
  <lastName>Smith</lastName>
  <age>25</age>
  <address>
    <streetAddress>21 2nd Street</streetAddress>
    <city>New York</city>
    <state>NY</state>
    <postalCode>10021</postalCode>
  </address>
  <phoneNumber>
    <type>home</type>
    <number>212 555-1234</number>
  </phoneNumber>
  <phoneNumber>
    <type>fax</type>
    <number>646 555-4567</number>
  </phoneNumber>
</person>
```

- **Tags**: `<element>value</element>` or `<element attribute="value"/>`
- **Repetition**: Multiple `<phoneNumber>` elements
- **Verbose**: More characters for structure

JSON:

```json
{"firstName": "John",
  "lastName": "Smith",
  "age": 25,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": "10021"
  },
  "phoneNumber": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "fax",
      "number": "646 555-4567"
    }
  ]
}
```

- **Objects**: `{}` for key-value pairs
- **Arrays**: `[]` for sequences
- **Compact**: Less verbose syntax

Working with XML and JSON in R

XML in R:

```r
library(xml2)

# Read XML file
xml_data <- read_xml("data/person.xml")

# Extract specific elements with Xpath
name <- xml_text(xml_find_all(xml_data, ".//firstName"))
phone_numbers <- xml_text(xml_find_all(xml_data, ".//phoneNum
phone_numbers <- xml_text(xml_children(xml_children(xml_data)

# Convert to data frame
person_df <- data.frame(
  firstName = name,
  phone = c(phone_numbers[1], phone_numbers[2])
)
```

```
  firstName        phone
1      John 212 555-1234
2      John 646 555-4567
```

JSON in R:

```r
library(jsonlite)

# Read JSON file
json_doc <- fromJSON("data/person.json")

# Access elements directly
name <- json_doc$firstName
phones <- json_doc$phoneNumber$number

# Convert to data frame (automatic!)
person_df <- as.data.frame(json_doc)
```

```
  firstName lastName age address.streetAddress
address.city address.state address.postalCode
1      John    Smith  25         21 2nd Street     New
York            NY              10021
2      John    Smith  25         21 2nd Street     New
York            NY              10021
  phoneNumber.type phoneNumber.number type
1             home       212 555-1234 male
2              fax       646 555-4567 male
```

HTML: Semi-structured Web Data

HTML documents/webpages consist of *'semi-structured data'*:

- A webpage can contain HTML tables (*structured data*)...
- ...but likely also contains just raw text (*unstructured data*).

Web scraping

- Extract data from HTML using XPath
- Example in class: parser for a Wikipedia table (webscraper) with `rvest`

Raw HTML structure:

```
<table class="wikitable">
<tbody>
<tr>
  <th>Year</th>
  <th>GDP (billions of
    <a href="/wiki/Swiss_franc">CHF</a>)
  </th>
  <th>US dollar exchange</th>
</tr>
<tr>
  <td>1980</td>
  <td>184</td>
  <td>1.67 Francs</td>
</tr>
...
</tbody>
</table>
```

REST APIs: Programmatic Data Access

REST APIs

- **Structured data**: APIs return data in JSON/XML format
- **HTTP requests**: Use GET, POST, PUT, DELETE methods
- **Real-time data**: Access current information (stock prices, weather, economic indicators)
- (**Authentication**: Often require API keys or tokens)
- **Rate limiting**: Restrictions on number of requests per time period

In R: Use `httr2`, `jsonlite` packages to consume APIs

Your group project

Use APIs to retrieve World Bank Data and OECD Data.

Warm-up

JSON files: open-ended question

You import the following json file using fromJSON() and store the file in the object students in R.

```
{
  "students": [
    {
      "id": 19091,
      "firstName": "Peter",
      "lastName": "Mueller",
      "grades": {
          "micro": 5,
          "macro": 4.5,
          "data handling": 5.5
          }
    },
    {
      "id": 19092,
      "firstName": "Anna",
      "lastName": "Schmid",
      "grades": {
          "micro": 5.25,
          "macro": 4,
          "data handling": 5.75
          }
    },
    {
```

- The object students is a list.
- `data$students$firstName[2]` returns “Anna”
- In JSON, the value for the “students” key is itself an object (key–value pair).
- In JSON, you have two ways of writing data: arrays and objects. Objects are key-value pairs.

```
{
  "name": "Anna",              // object (key-value pairs)
  "grades": [5, 5.5, 6]        // array (ordered list)
}
```

XML

```
<students>
  <student>
    <id>19091</id>
    <firstName>Peter</firstName>
    <lastName>Mueller</lastName>
    <grades>
      <micro>5</micro>
      <macro>4.5</macro>
      <dataHandling>5.5</dataHandling>
    </grades>
  </student>
  <student>
    <id>19092</id>
    <firstName>Anna</firstName>
    <lastName>Schmid</lastName>
    <grades>
      <micro>5.25</micro>
      <macro>4</macro>
      <dataHandling>5.75</dataHandling>
    </grades>
  </student>
  <student>
    <id>19093</id>
```

- ‘students’ is the root-node, ‘grades’ are its children
- The siblings of Trevor Noah are Anna Schmid and Peter Mueller
- This code would be an alternative, equivalent notation for the third student in the xml file above.

```
<student id="19093" firstName="Noah" lastName="Trevor">
      <grades micro="4" macro="4.5" dataHandling="5" />
</student>
```

Part II: Data preparation, analysis, and visualization

The dataset is imported, now what?

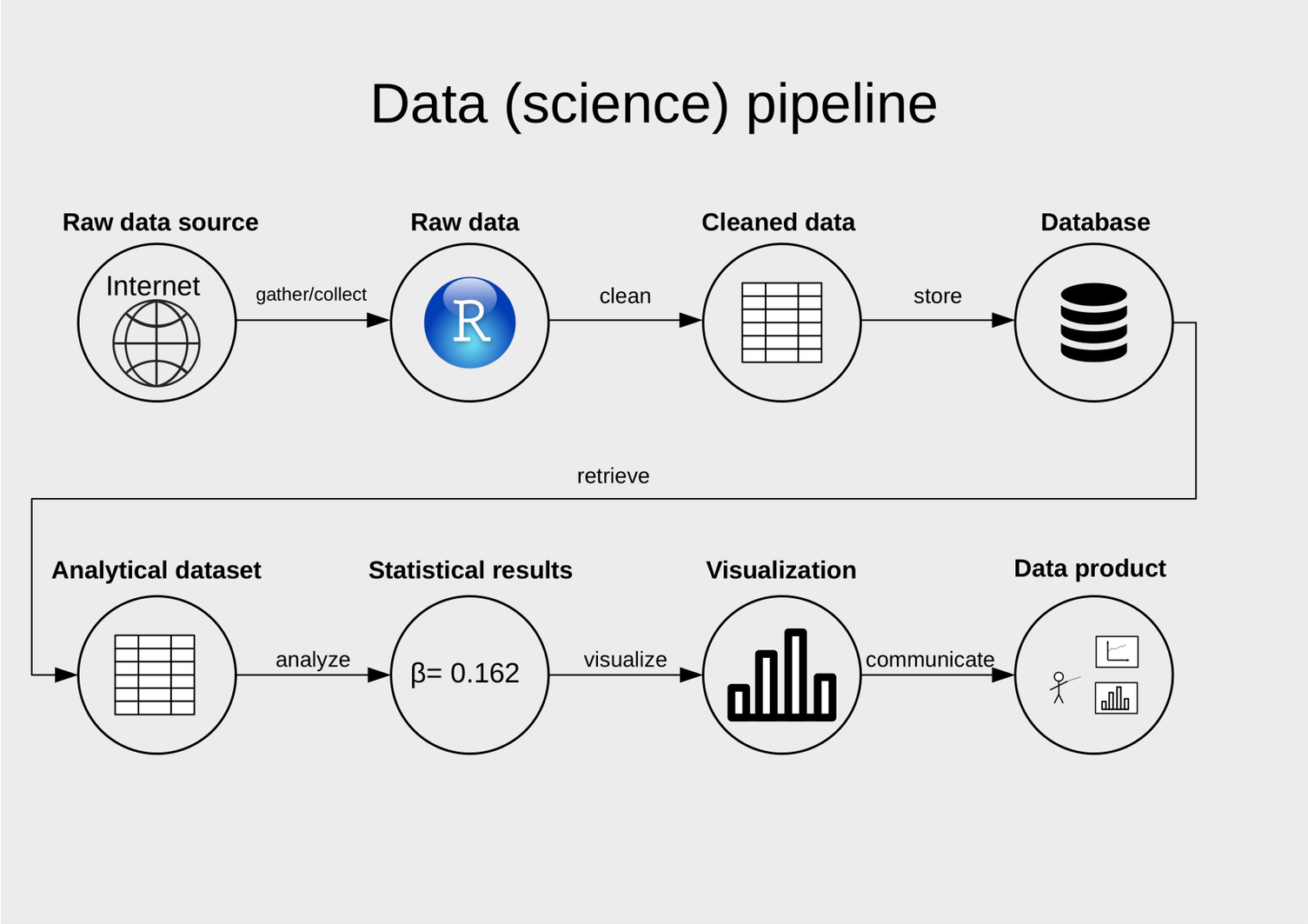
- In practice: still a long way to go.
- Parsable, but messy data: inconsistencies, data types, missing observations, wide format.

The dataset is imported, now what?

- In practice: still a long way to go.
- Parsable, but messy data: inconsistencies, data types, missing observations, wide format.

- **Goal** of data preparation: dataset is ready for analysis.

The dataset is imported, now what?



Part II: Schedule & Organization

Date	Topic
13.11.2025	Data preparation, manipulation, and cleaning I
20.11.2025	Data preparation, manipulation, and cleaning II
20.11.2025	Exercises/Workshop 4: Data gathering, data import
27.11.2025	Guest Lecture: Data Handling @Bank for International Settlements (Andrew Li, Ex-MiQeF, Macroeconic Analyst)

Part II: Schedule & Organization

Date	Topic
04.12.2025	Visualisation
04.12.2025	Exercises/Workshop 5: Data preparation and applied data analysis with R
11.12.2025	Analytics, more visualisation, and data products
11.12.2025	Short exam on the group project (in class)
18.12.2025	Summary, Wrap-up, Final workshop
18.12.2025	Exercises/Workshop 6: Visualization, dynamic documents
18.12.2025	Exam for Exchange Students

Part II: Schedule & Organization

Exam for exchange students 🎁

- **18.12.2024 at 16:15** in room 09-112 (library building).

Walk-ins for the digital exam:

- **Wednesday, 3 December 2025** from 10:30 to 14:00 in Room 23-104 (A 23 Lehr-Pavillon)
- **Tuesday, 16 December 2025** from 10:30 to 14:00 in 23-203 (A 23 Lehr-Pavillon)

Second part of the project: online today.

- Recommendation: wait for week 9 to start, as we will cover the necessary concepts in class until then.

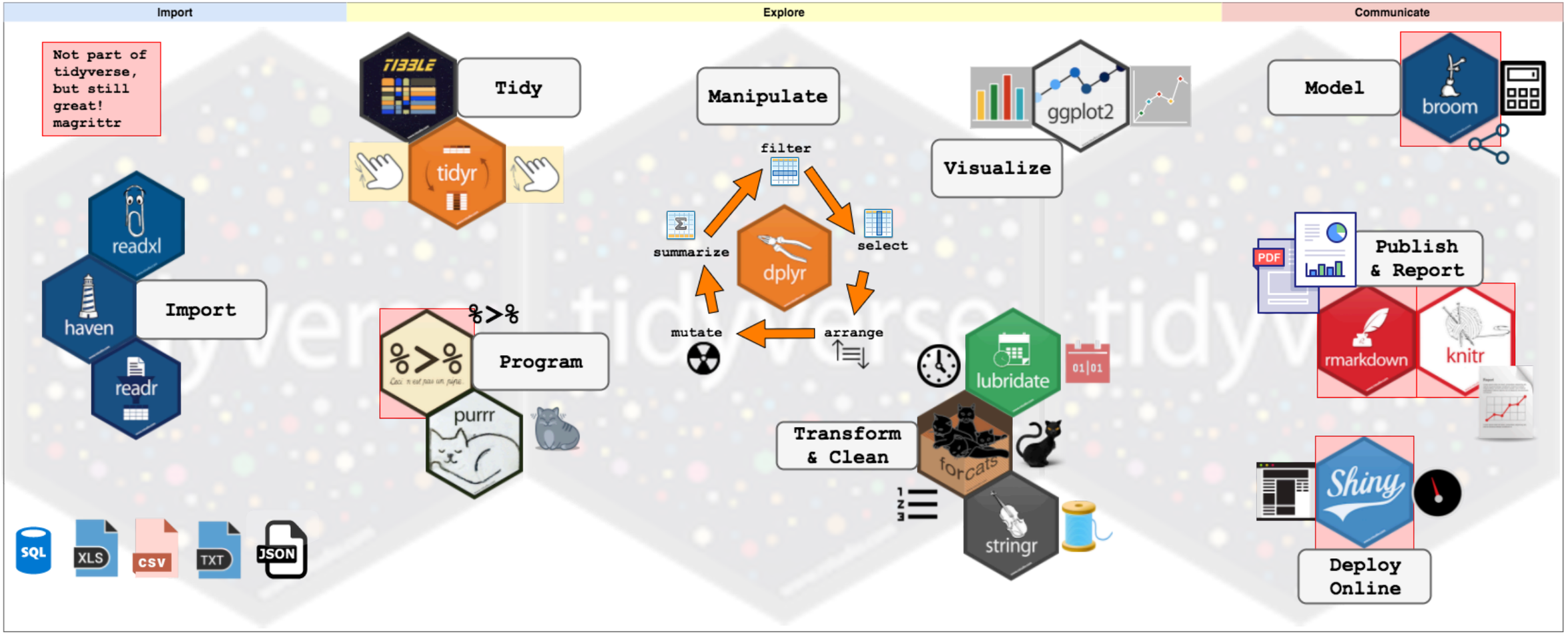
The Tidyverse

Our main tool in part II: the tidyverse

“The tidyverse is a collection of open source packages for the R programming language introduced by Hadley Wickham and his team that share an underlying design philosophy, grammar, and data structures” of tidy data.” (Wikipedia)

In this course, we will use tidyverse AND base R.

Tidyverse for data handling in R



Source: https://www.storybench.org/wp-content/uploads/2017/05/tidyverse.png

Rectangular data in R and tidyverse

- data frames: base `R`
- tibbles: `tidyverse`

A note on tibbles and data frames

Tibbles are a modern version of data frames

Similar!

- Tibbles are used in the tidyverse and ggplot2 packages.
- Same information as a data frame.
- Small differences in the manipulation and representation of data.
- See Tibble vs. DataFrame for more details.

Tibbles are a modern version of data frames

	Fertility	Agriculture	Examination	Education	Catholic	Infant.Mortality
Courtelary	80.2	17.0	15	12	9.96	22.2
Delemont	83.1	45.1	6	9	84.84	22.2
Franches-Mnt	92.5	39.7	5	5	93.40	20.2
Moutier	85.8	36.5	12	7	33.77	20.3
Neuveville	76.9	43.5	17	15	5.16	20.6
Porrentruy	76.1	35.3	9	7	90.57	26.6
Broye	83.8	70.2	16	7	92.85	23.6
Glane	92.4	67.8	14	8	97.16	24.9
Gruyere	82.4	53.3	12	7	97.67	21.0
Sarine	82.9	45.2	16	13	91.38	24.4
Veveyse	87.1	64.5	14	6	98.61	24.5
Aigle	64.1	62.0	21	12	8.52	16.5
Aubonne	66.9	67.5	14	7	2.27	19.1
Avenches	68.9	60.7	19	12	4.43	22.7
Cossonay	61.7	69.3	22	5	2.82	18.7
Echallens	68.3	72.6	18	2	24.20	21.2
Grandson	71.7	34.0	17	8	3.30	20.0
Lausanne	55.7	19.4	26	28	12.11	20.2
La Vallee	54.3	15.2	31	20	2.15	10.8
Lavaux	65.1	73.0	19	9	2.84	20.0
Morges	65.5	59.8	22	10	5.23	18.0
Moudon	65.0	55.1	14	3	4.52	22.4

Tibbles are a modern version of data frames

```
# A tibble: 47 × 6
   Fertility Agriculture Examination Education Catholic Infant.Mortality
       <dbl>       <dbl>       <int>     <int>    <dbl>             <dbl>
 1      80.2        17             15        12     9.96              22.2
 2      83.1        45.1            6         9    84.8               22.2
 3      92.5        39.7            5         5    93.4               20.2
 4      85.8        36.5           12         7    33.8               20.3
 5      76.9        43.5           17        15     5.16              20.6
 6      76.1        35.3            9         7    90.6               26.6
 7      83.8        70.2           16         7    92.8               23.6
 8      92.4        67.8           14         8    97.2               24.9
 9      82.4        53.3           12         7    97.7               21
10      82.9        45.2           16        13    91.4               24.4
# ℹ 37 more rows
```

Data cleaning

Garbage in, garbage … ?

Beware of the “Garbage in garbage out” problem

Key conditions:

1. Data values are consistent/clean within each variable.
2. Variables are of proper data types.
3. Dataset is in ‘tidy’ (long) format.

Data preparation can be understood as consisting of five main actions

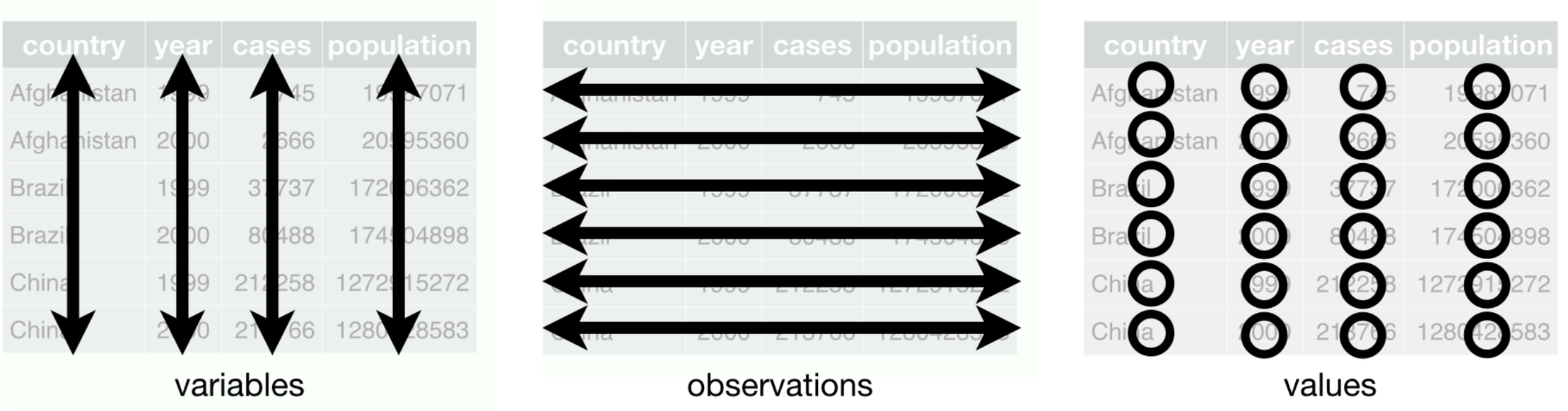
- **Tidy** data.
- **Reshape** datasets from wide to long (and vice versa).
- **Bind** or stack rows in datasets.
- **Join** datasets.
- **Clean** data.

Tidy data: some vocabulary

Following R4DS, a tidy dataset is tidy when...

1. Each variable is a column; each column is a variable.
2. Each observation is a row; each row is an observation.
3. Each value is a cell; each cell is a single value.

Tidy data



Tidy data. Source R4DS.

Tidy data is not trivial

In Economics, the definition of an *observation* can vary:

- Panel data (longitudinal data)
- Cross-sectional data
- Time series

Tidy data is not trivial

Panel Data (Longitudinal Data):

- Panel data tracks the same units over multiple time periods: we observe each unit repeatedly across time.
- **Observation**: One unit at one specific time point (e.g., Person A in 2020, Person A in 2021).

Cross-Sectional Data:

- “Snapshot” of different units at the same moment.
- **Observation**: One unit at a single point in time (e.g., Person A, Person B, Person C all measured in 2024).

Time Series Data:

Three examples of non-tidy data (1)

Messy 💩

```
# A tibble: 2 × 4
  measure     `Jan 1` `Jan 2` `Jan 3`
  <chr>         <dbl>   <dbl>   <dbl>
1 Temperature      20      22      21
2 Humidity         80      78      82
```

Tidy 😎

...

Three examples of non-tidy data (1)

Messy 💩

```
# A tibble: 2 × 4
  measure     `Jan 1` `Jan 2` `Jan 3`
  <chr>         <dbl>   <dbl>   <dbl>
1 Temperature      20      22      21
2 Humidity         80      78      82
```

Tidy 😎

```
# A tibble: 3 × 3
  Date  Temperature Humidity
  <chr>       <dbl>    <dbl>
1 Jan 1          20       80
2 Jan 2          22       78
3 Jan 3          21       82
```

Three examples of non-tidy data (2)

Messy 💩

```
# A tibble: 3 × 2
   year temperature_location
  <dbl> <chr>
1  2019 22C_London
2  2019 18C_Paris
3  2019 25C_Rome
```

Tidy 😎

```
homework..
```

Three examples of non-tidy data (3)

Messy 💩

```
   Student Econ DataHandling Management
1 Johannes 5.00          4.0        5.5
2   Hannah 5.25          4.5        6.0
3     Igor 4.00          5.0        6.0
```

Tidy 😎

```
homework..
```

Reshaping

Reshaping: the concept

"Long" format

country	year	metric
x	1960	10
x	1970	13
x	2010	15
y	1960	20
y	1970	23
y	2010	25
z	1960	30
z	1970	33
z	2010	35

"Wide" format

country	yr1960	yr1970	yr2010
x	10	13	15
y	20	23	25
z	30	33	35

Long and wide data. Source: Hugo Tavares

Wide vs Long Format: Key Differences

Long Format

- **Structure**: Each measurement = one row
- **Variables**: Stacked vertically
- **Dimensions**: More rows, fewer columns
- **Analysis**: Better for statistical operations
- **Example**: Database-normalized format

```
# A tibble: 9 × 3
  Student  Subject      Grade
  <chr>    <chr>        <dbl>
1 Johannes Econ           5
2 Johannes DataHandling   4
3 Johannes Management     5.5
4 Hannah   Econ           5.25
5 Hannah   DataHandling   4.5
6 Hannah   Management     6
7 Igor     Econ           4
8 Igor     DataHandling   5
9 Igor     Management     6
```

Wide Format

- **Structure**: Each observation = one row
- **Variables**: Spread across multiple columns
- **Dimensions**: More columns, fewer rows
- **Analysis**: Easy for humans to read
- **Example**: Spreadsheet-style layout

```
   Student Econ DataHandling Management
1 Johannes 5.00          4.0        5.5
2   Hannah 5.25          4.5        6.0
3     Igor 4.00          5.0        6.0
```

Reshaping: implementation in R

- From wide to long: `melt()`, `gather()`,

👉 We'll use `tidyverse::pivot_longer()`.

- From long to wide: `cast()`, `spread()`,

👉 We'll use `tidyverse::pivot_wider()`.

Reshaping: implementation in R with `tidyverse()`

country	year	metric
x	1960	10
x	1970	13
x	2010	15
y	1960	20
y	1970	23
y	2010	25
z	1960	30
z	1970	33
z	2010	35

```
pivot_wider(names_from = “year”,
            names_prefix = “yr”,
            values_from = “metric”)
```

country	yr1960	yr1970	yr2010
x	10	13	15
y	20	23	25
z	30	33	35

```
pivot_longer(cols = yr1960:yr2010,
             names_to = “year”,
             names_prefix = “yr”
             values_to = “metric”)
```

When to Use Wide vs Long Format

Use Wide Format For:

- **Data entry** and manual input
- **Reports and tables** for presentation
- **Some statistical models** (e.g., repeated measures ANOVA)
- **Spreadsheet analysis** (Excel-style operations)
- **Human readability** and quick scanning

Use Long Format For:

- **Data analysis** with tidyverse/ggplot2
- **Statistical modeling** (most R functions)
- **Grouping operations** (summarize, mutate by groups)
- **Time series analysis**
- **Following tidy data principles**
- **Creating visualizations** with multiple variables

Rule of thumb: Wide format for presentation, long format for analysis

Stack and row-bind

Stack/row-bind: the concept

ID	X	Y
1	a	50
2	b	10

ID	Z
3	M
4	O

ID	X	Z
5	c	P

ID	X	Y	Z
1	a	50	NA
2	b	10	NA
3	NA	NA	M
4	NA	NA	O
5	c	NA	P

Stack/row-bind: implementation in R

- Use `rbind()` in base `R`
  - Requires that the data frames have the same column names and same column classes.

- Use `bind_rows()` from `dplyr()`
  - More flexible
  - Binds data frames with different column names and classes
  - Automatically fills missing columns with `NA`

For these reasons (+ performance, handling or row names, and handling of factors), `dplyr::bind_rows()` *is preferred in most applications.*

Stack/row-bind: code example

```
# Create three dfs
subset1 <- data.frame(ID = c(1,2),
                      X = c("a", "b"),
                      Y = c(50,10))

subset2 <- data.frame(ID = c(3,4),
                      Z = c("M", "O"))

subset3 <- data.frame(ID = c(5),
                      X = c("c"),
                      Z = "P")

# Inspect
subset1
```

```
  ID X  Y
1  1 a 50
2  2 b 10
```

```
subset2
```

```
  ID Z
1  3 M
2  4 O
```

```
subset3
```

```
  ID X Z
1  5 c P
```

Stack/row-bind: code example

```
# Stack data frames
combined_df_bind_rows <- bind_rows(subset1, subset2, subset3)
combined_df_rbind     <- rbind(subset1, subset2, subset3)
```

```
Error in rbind(deparse.level, ...): numbers of columns of arguments do not match
```

```
# What are the following objects?
combined_df_bind_rows
```

```
  ID    X  Y    Z
1  1    a 50 <NA>
2  2    b 10 <NA>
3  3 <NA> NA    M
4  4 <NA> NA    O
5  5    c NA    P
```

```
combined_df_rbind
```

```
Error: object 'combined_df_rbind' not found
```

The errors are due to the requirements of `rbind()`.

Appendix

Tibbles vs data frames

- Unlike data frames, tibbles don’t show the entire dataset when you print it
- Tibbles cannot access a column when you provide a partial name of the column, but data frames can.
- When you access only one column of a tibble, it will keep the tibble structure. But when you access one column of a data frame, it will become a vector.
- When assigning a new column to a tibble, the input will not be recycled, which means you have to provide an input of the same length of the other columns. But a data frame will recycle the input.
- Tibbles preserve all the variable types, while data frames have the option to convert string into factor. (In older versions of R, data frames will convert string into factor by default)

Source: https://jtr13.github.io/cc21fall1/tibble-vs.-dataframe.html